

Python Topic 1 — Variables & Data Types

Full Stack Python + AI Roadmap • Taught in 3 layers: New to Python → Intermediate → Expert



Layer 1 — New to Python

A **variable** is just a name you give to a value so you can use it later. You create one by writing a name, an equals sign, and the value:

```
toy = "car"
```

Now the name `toy` refers to the text `"car"`. Whenever you write `toy`, Python gives you back `"car"`.

You don't have to tell Python what *kind* of value it is. Python looks at the value and figures it out automatically — put a number and it's a number, put text and it's text:

```
age = 26           # Python knows this is a number
name = "Akshay"    # Python knows this is text
print(name)        # shows: Akshay
```

That's the whole idea to start: a name on the left, a value on the right, and Python remembers it for you.



Layer 2 — Intermediate (real code + how it works)

Python is **dynamically typed** — you don't declare types, the value decides. There's no `let` or `const` keyword; the variable is created on assignment.

```
name = "Akshay"      # str
age = 26              # int
height = 5.9          # float
is_married = True     # bool
skills = ["React", "Python"] # list
profile = {"role": "dev"} # dict
```

JavaScript → Python

```
// JS
let name = "Akshay";
const age = 26;

# Python — no let/const
```

```
name = "Akshay"
age = 26
```

Python has no `const`. Convention: `ALL_CAPS` means "treat as a constant" — but nothing actually stops reassignment.

```
MAX_RETRIES = 3  # by convention, a constant
```

i Why use ALL_CAPS? It's a signal to other developers: "this value is meant to stay fixed." Use it for config & fixed limits (`MAX_RETRIES` , `API_URL` , `TIMEOUT`) so "magic numbers" get a readable name. Unlike JS `const` (which throws on reassignment), Python's CAPS is a **pinky promise, not a padlock** — nothing stops reassignment.

Core types you'll use daily

Type	Example	JS Equivalent
<code>int</code>	<code>42</code>	<code>number</code>
<code>float</code>	<code>3.14</code>	<code>number</code>
<code>str</code>	<code>"hi"</code>	<code>string</code>
<code>bool</code>	<code>True</code> / <code>False</code>	<code>true</code> / <code>false</code>
<code>list</code>	<code>[1, 2, 3]</code>	<code>Array</code>
<code>dict</code>	<code>{"k": "v"}</code>	<code>Object</code> / <code>Map</code>
<code>tuple</code>	<code>(1, 2)</code>	frozen array (no direct equiv)
<code>None</code>	<code>None</code>	<code>null</code> / <code>undefined</code>

Layer 3 — Expert (internals & gotchas)

1. Everything is an object — even integers

In Python, **every value is an object**: numbers, strings, lists, dicts, booleans, `None`, functions, even classes and modules. Each object carries three things — a **value**, a **type**, and an **identity** (memory address). A variable is just a **name** pointing to that object, not the object itself.

```
x = 42
print(x.bit_length())  # 6 → yes, an int has methods!
print(type(x))         # <class 'int'> → its type
print(id(x))           # 140352... → its identity
```

i JS contrast: In JavaScript, `42` is a **primitive** (a raw value). In Python there are no primitives — `42` is a genuine object with methods, all the time. Numbers/strings just *feel* primitive because they're immutable.

Because variables are names → objects (not containers holding bytes), identity caching becomes visible:

```
x = 256
y = 256
print(x is y)    # True

a = 257
b = 257
print(a is b)    # False (!) — CPython caches only -5 to 256
```

i Why: CPython pre-allocates small integers (`-5` to `256`) as singletons for performance. Outside that range they're distinct objects. Always compare **values with** `==`, and **identity with** `is` (use `is` only for `None`, `True`, `False`).

🔗 Memory hook: Names are luggage tags; objects are the suitcases. Two tags can hang on the same suitcase.

2. The mutable default trap (classic interview killer)

```
def add(item, bucket=[]):    # 🚩 shared across all calls!
    bucket.append(item)
    return bucket

add(1)    # [1]
add(2)    # [1, 2] — not [2]!
```

🚩 Trap: The default `[]` is created **once** at function definition, not per call. Every call without an argument shares the same list.

Fix — use `None` as the sentinel:

```
def add(item, bucket=None):
    if bucket is None:
        bucket = []
    bucket.append(item)
    return bucket
```

3. Names vs values (the reference model)

```
a = [1, 2, 3]
b = a    # b points to the SAME list
```

```
b.append(4)
print(a)      # [1, 2, 3, 4] – a changed too!
```

This mirrors JS objects/arrays being passed by reference — but Python applies the model to **everything**. Assignment never copies; it only binds a new name to an existing object.

Interview Q&A Cards

Q: Are Python variables typed?

A: No. Variables are untyped *names* bound to typed *objects*. The object carries the type; the name is just a reference. This is dynamic typing.

Q: What does "everything is an object" mean in Python?

A: Every value — numbers, strings, lists, functions, even classes — is an object carrying a value, a type, and an identity. Variables are names pointing to these objects; assignment moves the name, never copies the object.

Q: Difference between `==` and `is` ?

A: `==` compares values (equality); `is` compares identity (same object in memory). Use `is` only for `None`, `True`, `False`.

Q: Why is a mutable default argument dangerous?

A: It's evaluated once at function-definition time and shared across all calls, so state leaks between invocations. Use `None` as a sentinel and create the object inside the function.

 **Memory hook:** *Variables are sticky notes, not boxes.* Two sticky notes can point at the same toy — change the toy through one, the other sees it too.

✓ **One-liner for interviews:** "Python assignment binds a name to an object; it never copies. Mutability of the **object** — not the variable — determines behavior."